HW03 — Linear and Logistic Regression

Specification

Ramaz ML Course 2026

This document is the **contract** for HW03. It tells you exactly what to build, what each method must do, and the shape of the public API your code must match. The test suite checks this contract directly — when your code matches the spec, the tests pass.

How to use this document: Read it once end-to-end before opening `regression.py`. Then implement methods in roughly the order tests are written (method tests first, then `fit`, then anything left). Refer back here whenever you forget a math formula or a return shape.

Contents

1 Architecture overview **2**

2 Standalone helper: `sigmoid` **2**

3 Base class: `LinearModel` **2**
- 3.1 `__init__` . . . 3
- 3.2 `gradient` . . . 3
- 3.3 `fit` . . . 4
- 3.4 `predict` . . . 4

4 LinearRegression **4**
- 4.1 `forward` . . . 5
- 4.2 `loss` . . . 5

5 LogisticRegression **5**
- 5.1 `forward` . . . 6
- 5.2 `loss` . . . 6
- 5.3 `predict_proba` . . . 6
- 5.4 `predict` . . . 6

6 Analysis (`analysis.py`) **7**
- 6.1 `load_regression_data` . . . 7
- 6.2 `train_linear_model` . . . 7
- 6.3 `load_classification_data` . . . 7
- 6.4 `majority_class_baseline` . . . 8
- 6.5 `train_logistic_model` . . . 8

7 How to read the test file **8**

8 Restrictions **9**

1 Architecture overview

Linear regression and logistic regression are almost the same algorithm: both predict from a linear combination $X\mathbf{w} + b$, both train by full-batch gradient descent, and — with the loss conventions in this spec — both use the *same* parameter gradient. You capture that shared structure in one base class, `LinearModel`, and let each model override only what genuinely differs.

Base class `LinearModel` holds the machinery common to both models: the parameters and learning rate (`__init__`), the gradient (`gradient`), the training loop (`fit`), and a default `predict`. The two concrete models subclass it and differ only as follows:

	`LinearRegression`	`LogisticRegression`
`forward`	$X\mathbf{w} + b$	$\sigma(X\mathbf{w} + b)$
`loss`	MSE (with the $\frac{1}{2}$ factor)	binary cross-entropy
`predict`	inherited (= `forward`)	hard 0/1 label (override)
`predict_proba`	—	soft probability (= `forward`)

Every training iteration runs the same five steps:

1. `forward(X)` computes predictions $\hat{\mathbf{y}}$ from inputs X.
2. `loss(y_pred, y_true)` returns a scalar loss.
3. `gradient(X, y_true, y_pred)` returns (dw, db).
4. Update step: $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot dw, \quad b \leftarrow b - \eta \cdot db$.
5. Repeat for `epochs` iterations.

Steps 3–5 live in `LinearModel` and are shared; steps 1–2 are the subclass methods. `fit(X, y, epochs)` runs this loop `epochs` times and returns the list of loss values, one per epoch.

2 Standalone helper: `sigmoid`

You must implement one free function before either class:

```
def sigmoid(z:  torch.Tensor) -> torch.Tensor
```

Apply this activation to the logistic model's linear combination to turn each logit into a probability.

Element-wise sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output has the same shape as the input. In exact arithmetic it lies in $(0, 1)$ and satisfies $\sigma(-z) = 1 - \sigma(z)$. Floating-point values can round to 0 or 1 for large logits.

3 Base class: `LinearModel`

`LinearModel` is the shared base. `LinearRegression` and `LogisticRegression` both inherit from it, so you write each of the methods below *once*. Match every signature exactly — the tests construct the subclasses and call these inherited methods on them.

3.1 `__init__`

```
def __init__(self, n_features:  int, lr:  float = 0.1) -> None
```

Set up a fresh model. Called once, when the user does `model = LinearRegression(n_features=8)`. After this, the model has parameters initialized to zero but hasn't seen any data yet.

Initialize:

- `self.w` — a 1-D tensor of zeros with shape `(n_features,)`, dtype `torch.float32`.
- `self.b` — a scalar tensor (0-D) with value 0.0, dtype `torch.float32`.
- `self.lr` — store the learning rate as-is.

The grader reads `self.w` and `self.b` after `fit`, so use these exact attribute names.

A note on 0-D (scalar) tensors. The bias b is a single number, but it must be a *tensor*, not a Python float, so the update step $b \leftarrow b - \eta\, db$ stays a tensor operation:

- Create one with `torch.tensor(0.0)`. Beware: `torch.tensor([0.0])` gives a *1-D* tensor of length one, which is not the same thing.
- A 0-D tensor has `t.dim() == 0` and `t.shape == torch.Size([])`.
- Reducing a vector to a single number yields a 0-D tensor: `r.sum()` is 0-D. Read its Python value with `t.item()`.
- Scalar arithmetic preserves the shape: if `b` and `db` are both 0-D, then `b - lr * db` is still 0-D.

The tests check `self.b.dim() == 0`, so keep b scalar throughout.

3.2 `gradient`

```
def gradient(self, X: torch.Tensor, y_true:  torch.Tensor, y_pred:  torch.Tensor)
    -> tuple[torch.Tensor, torch.Tensor]
```

Step 3 of every training iteration. Figure out which direction to move each parameter to reduce the loss. The training loop will subtract a small multiple (`lr`) of these gradients from $\mathbf{w}$ and b.

Compute the gradient of the loss with respect to $\mathbf{w}$ and b. Use the formula derived in lecture (no `torch.autograd`, no `.backward()`):

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{1}{n} X^\top (\hat{\mathbf{y}} - \mathbf{y}) \qquad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

The residual is $\hat{\mathbf{y}} - \mathbf{y}$ = `y_pred - y_true` (predictions minus targets). Mind the order: `y_true` appears *before* `y_pred` in the signature, but the residual subtracts `y_true` from `y_pred`.

This single method lives in the base class and serves *both* subclasses: with the loss conventions in this spec, the gradient of the MSE loss (§4) and the gradient of mathematical binary cross-entropy (§5) reduce to the same expression. Use the original predictions in the residual. The clipping in §5 keeps the reported loss finite. The regression lecture derives both gradients, and writeup Question 3 asks you to reproduce the cancellations.

- `dw` has shape `(d,)`.
- `db` is a scalar (0-D) tensor.

3.3 `fit`

```
def fit(self, X: torch.Tensor, y:  torch.Tensor, epochs:  int = 200) -> list[float]
```

Run the full training loop. This is the only method a normal user will call directly — it orchestrates `forward`, `loss`, `gradient`, and the parameter update step in sequence. Because it calls `self.forward` and `self.loss`, the *same* loop trains whichever subclass it is running on.

Full-batch gradient descent. The algorithm:

1. Initialize an empty history list.
2. Repeat for `epochs` iterations:
   (a) Compute predictions from the current parameters (call your own `forward`).
   (b) Compute the loss between predictions and targets (call your own `loss`).
   (c) Append the loss *as a Python float* (not a tensor) to the history.
   (d) Compute the gradients (call your own `gradient`).
   (e) Update parameters by stepping against the gradient: $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot dw, \quad b \leftarrow b - \eta \cdot db.$
3. Return the history list.

- Returns a `list[float]` of length exactly `epochs`.
- After `fit` returns, `self.w` and `self.b` hold the trained parameters.

3.4 `predict`

```
def predict(self, X: torch.Tensor) -> torch.Tensor
```

Inference. Produce predictions on new data using the trained parameters. The base-class version simply returns `forward(X)` — the correct behavior for regression, where the prediction *is* the linear output. `LogisticRegression` overrides this to return a hard class label (see §5).

Invariants the tests will check

- `gradient` returns `(dw, db)` with shapes `(d,)` and scalar (0-D).
- For sufficiently small `lr`, the loss history is non-increasing.
- `fit` returns a `list[float]` of length `epochs`; after it returns, `self.w` and `self.b` hold the trained parameters.

4 LinearRegression

`LinearRegression(LinearModel)` inherits `__init__`, `gradient`, `fit`, and `predict` from §3. You implement only the two methods that make it linear regression: `forward` and `loss`.

4.1 `forward`

```
def forward(self, X: torch.Tensor) -> torch.Tensor
```

Step 1 of every training iteration. Take the current parameters and produce a prediction for each row of X. Also called at inference time (via the inherited `predict`) on new data.

Compute the linear prediction:

$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

- Input shape: `X` has shape `(n, d)` where `d == self.w.shape[0]`.
- Output shape: `(n,)`.

4.2 `loss`

```
def loss(self, y_pred:  torch.Tensor, y_true:  torch.Tensor) -> torch.Tensor
```

Step 2 of every training iteration. Collapse the per-example prediction error into one scalar. The training loop tries to make this number smaller.

Mean squared error, with the conventional factor of $\frac{1}{2}$:

$$\text{MSE} = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

The $\frac{1}{2}$ is a convention, not a different loss — it scales the loss by a constant and so does not change where the minimum is. It is chosen so that the derivative of this loss comes out in the clean form the base-class `gradient` uses for both models (writeup Question 3 asks you to verify this).

- Inputs: both shape `(n,)`.
- Output: a 0-dimensional (scalar) tensor.

Invariants the tests will check

- On data generated by $y = Xw_{\text{true}} + b_{\text{true}} + \varepsilon$ with small ε, `fit` for enough epochs recovers w_{true} and b_{true} to within tolerance.
- `forward(X)` returns shape `(n,)`; the inherited `predict(X)` returns the same thing as `forward(X)`.

5 LogisticRegression

`LogisticRegression(LinearModel)` also inherits `__init__`, `gradient`, and `fit` from §3. You implement `forward` and `loss`, add `predict_proba`, and *override* `predict` (a hard label instead of the raw score the base class returns). Match each signature exactly.

5.1 `forward`

```
def forward(self, X: torch.Tensor) -> torch.Tensor
```

Step 1 of every training iteration. Produce a class-1 probability for each input row.

Probability of class 1:

$$\hat{\mathbf{y}} = \sigma(X\mathbf{w} + b)$$

Use the standalone `sigmoid` you implemented in §2.

- Output shape: `(n,)`. Values lie in $[0, 1]$ in floating point.

5.2 `loss`

```
def loss(self, y_pred:  torch.Tensor, y_true:  torch.Tensor) -> torch.Tensor
```

Step 2 of every training iteration. Measure prediction error for a binary classification problem. BCE penalizes confident wrong predictions much more than uncertain ones.

Binary cross-entropy:

$$\text{BCE} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

- `y_true` values are in $\{0, 1\}$ (encoded as floats).
- **Numerical safety:** clamp `y_pred` into $[\varepsilon, 1 - \varepsilon]$ with $\varepsilon = 10^{-7}$ before taking `log`. Otherwise $\log(0) = -\infty$ will blow up your loss.

5.3 `predict_proba`

```
def predict_proba(self, X: torch.Tensor) -> torch.Tensor
```

Inference, soft version. Returns the probability of class 1 without thresholding. Useful when callers want a confidence value.

Returns the same vector of probabilities as `forward(X)`. Callers can read soft predictions before applying a threshold.

5.4 `predict`

```
def predict(self, X: torch.Tensor) -> torch.Tensor
```

Inference, hard version. Overrides the base-class `predict` to return the predicted class label (0 or 1) for each input.

Apply the standard 0.5 threshold:

$$y_i = \begin{cases} 1 & \text{if } \hat{y}_i > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

- Output shape: `(n,)`, values in $\{0, 1\}$.
- Output dtype should be `torch.float32`, so it compares equal to the float-encoded 0/1 labels in our datasets.

Invariants the tests will check

- `forward` outputs are probabilities in $[0, 1]$.
- On linearly separable data, `predict` achieves $> 95\%$ accuracy after enough epochs.
- `predict` returns hard labels in $\{0, 1\}$: an example is labeled class 1 exactly when its predicted probability exceeds one-half, and class 0 otherwise.

6 Analysis (`analysis.py`)

`analysis.py` contains five utility functions used by the test suite and by your final `analysis.py` run that produces plots for `writeup.md`. Four of them are data-loading and training wrappers — the learning goal is the regression classes above, not data preprocessing — so the spec names the sklearn entry points directly. The fifth, `majority_class_baseline`, is the yardstick you measure your classifier against. Match the signatures exactly.

6.1 `load_regression_data`

```
def load_regression_data() -> tuple[Tensor, Tensor, Tensor, Tensor]
```

Load the California Housing dataset and return train/test splits as float32 tensors.

Steps:

1. Call `sklearn.datasets.fetch_california_housing()`.
2. Split with `sklearn.model_selection.train_test_split(test_size=0.2, random_state=42)`.
3. Standardize features: fit a `StandardScaler` on the training set only, then transform both sets.
4. Convert all four arrays to `torch.Tensor` with `dtype=torch.float32`.

Return order: `(X_train, X_test, y_train, y_test)`. Shapes: `X_train` is `(n_train, 8)`, `X_test` is `(n_test, 8)`, labels are 1-D.

6.2 `train_linear_model`

```
def train_linear_model(X_train:  Tensor, y_train:  Tensor,
    lr:  float = 0.01, epochs:  int = 200) -> tuple[LinearRegression, list[float]]
```

Thin wrapper: construct a fresh `LinearRegression` with `n_features=X_train.shape[1]` and the given `lr`, call its `fit` method for `epochs` epochs, and return `(model, loss_history)`.

6.3 `load_classification_data`

```
def load_classification_data() -> tuple[Tensor, Tensor, Tensor, Tensor]
```

Load a 2-class subset of the Iris dataset (setosa vs. versicolor), keeping only the first two features so the decision boundary can be plotted in 2D.

Steps:

1. Call `sklearn.datasets.load_iris()`.

2. Filter to rows where `target < 2`; keep only the first two feature columns.
3. Split with `train_test_split(test_size=0.2, random_state=42)`.
4. Standardize features (fit on train only).
5. Convert all four arrays to `float32` torch tensors.

Labels are in $\{0, 1\}$ encoded as floats.

6.4 `majority_class_baseline`

```
def majority_class_baseline(y:  torch.Tensor) -> float
```

Before you can call a classifier good, you need to know what a classifier that ignores all features would score. This function returns the fraction of labels belonging to the most common class in `y`. The provided experiment driver uses it to describe the training split, then evaluates the same constant prediction on the test split before training the model (writeup Question 4).

Let n be the number of labels and let n_c be how many of them equal class c. Always guessing class c is correct exactly on the n_c examples that really are class c, so guessing the most common class gives

$$\text{acc}_{\text{majority}} = \frac{\max_c n_c}{n}.$$

Note that this depends on the *labels only* — two datasets with the same label counts have the same baseline no matter how separable their features are.

- Input: `y` is a 1-D tensor of class labels (float-encoded $\{0, 1\}$ here), with at least one element.
- Returns a Python `float`, not a tensor.
- The value lies in $(0, 1]$, and for two classes it is never below 0.5: whichever class is in the minority, the other one covers at least half the data.

6.5 `train_logistic_model`

```
def train_logistic_model(X_train:  Tensor, y_train:  Tensor,
    lr:  float = 0.5, epochs:  int = 300) -> tuple[LogisticRegression, list[float]]
```

Same shape as `train_linear_model` — construct a fresh `LogisticRegression`, call `fit`, return `(model, loss_history)`.

7 How to read the test file

`test_regression.py` is organized in three layers — work on them in order:

`@pytest.mark.methods` — one method at a time

Each test class targets one piece of one class. Get these green first; they verify the math primitives independently (forward shapes, loss values, gradients matching the formula) without requiring a working training loop.

`@pytest.mark.pipeline` — `fit` end-to-end

Once the method tests pass, move on. Pipeline tests run a full `fit` on synthetic data and check that the loss decreases, that the recovered parameters are close to ground truth, and that `predict` produces sensible outputs.

`@pytest.mark.experiments` — `analysis.py` helpers

Tests for the five helper functions in `analysis.py` (data loading, training wrappers, and the majority-class baseline).

Run a single layer with:

```
uv run pytest -m methods        # just method tests
uv run pytest -m pipeline       # just pipeline tests
uv run pytest -m experiments    # just experiment-helper tests
```

8 Restrictions

- **No `torch.autograd`, no `.backward()`, no `requires_grad=True` anywhere in `regression.py`.** You are computing every gradient by hand from the derivations in lecture. The `conftest.py` fails the run if it detects autograd usage in your file.
- **No built-in sigmoid in `regression.py`.** You implement `sigmoid` yourself from the formula in §2. `torch.sigmoid`, `torch.nn.functional.sigmoid`, and `torch.special.expit` are blocked by `conftest.py`. Built-in tensor methods such as `z.sigmoid()` and `z.sigmoid_()` are also blocked, including unbound calls such as `torch.Tensor.sigmoid(z)`.
- **No `sklearn.linear_model`.** Same reason — implementing logistic regression yourself is the point of the assignment.
- Use PyTorch for array operations. The data-loading utilities may return arrays, which you can index and convert to tensors.